LAPRAK TEKNOLOGI IOT II

PRATIKUM 7

Oleh:

NAMA : M. Kurnia Akbar
NPM : 25782101
KELAS : TRI – 3D

Dosen:
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.
Andi Saptono, S.Kom

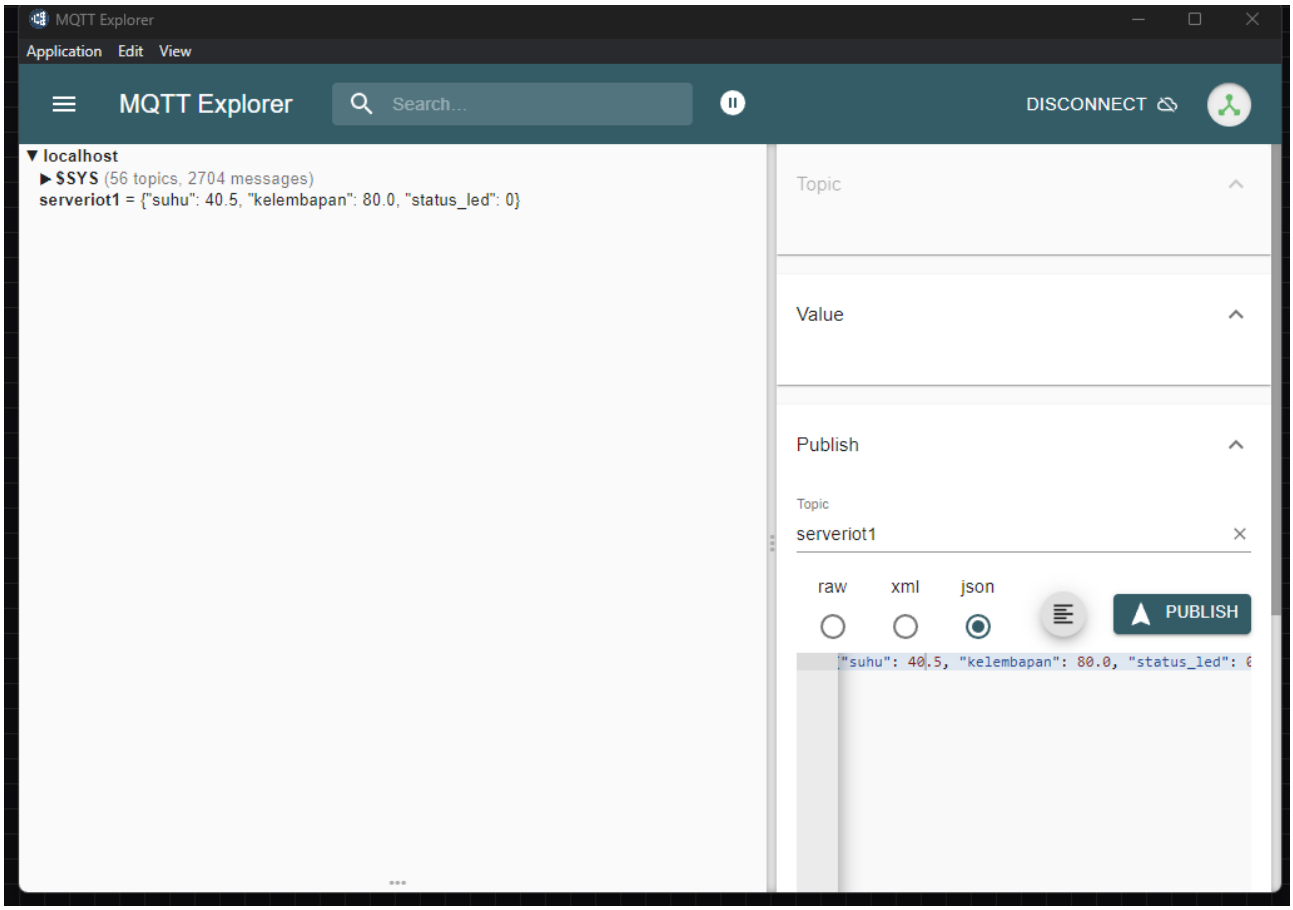
TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

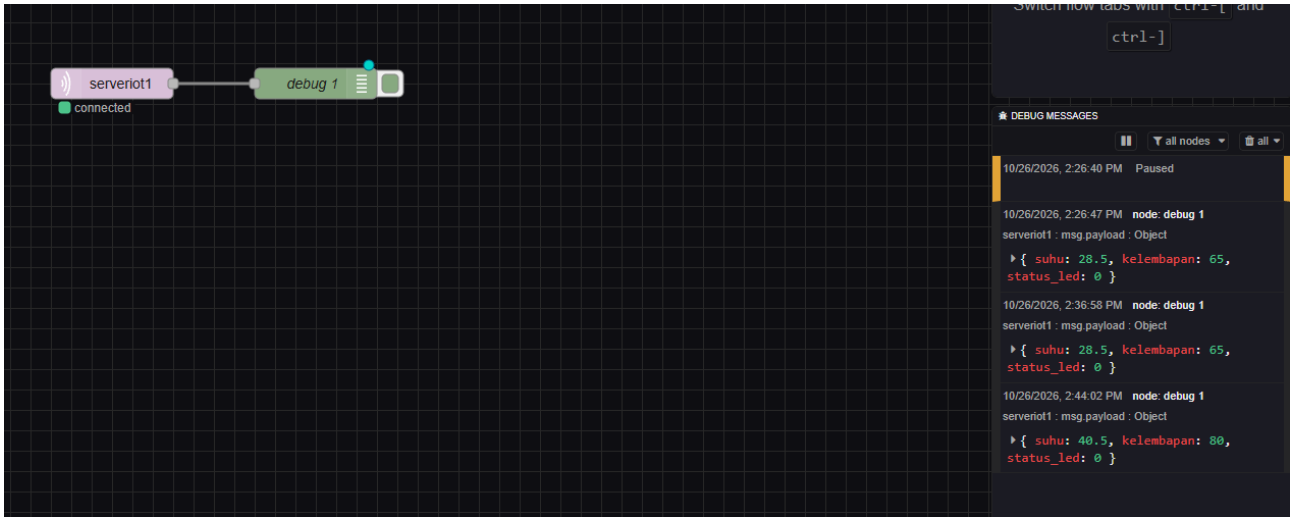
2025/2026

Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.

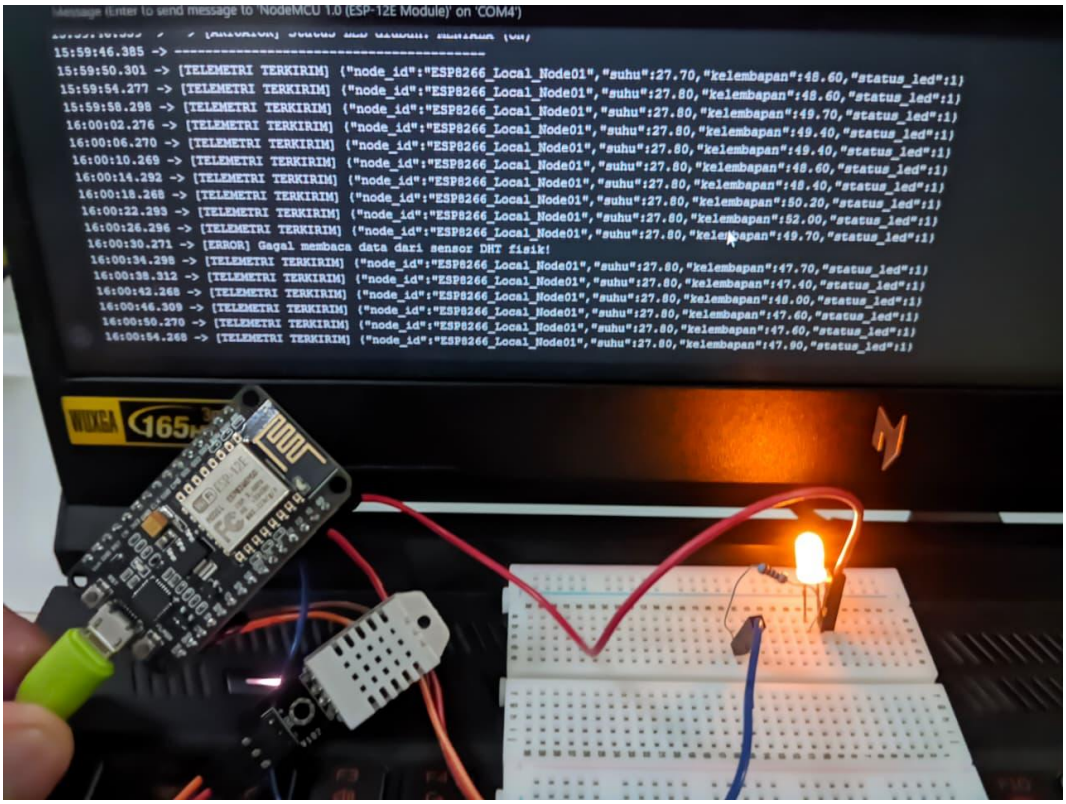
```
D:\Kuliah\iot-server>docker compose up -d
[+] up 2/2
 ✔Container node-red     Running
 ✔Container mqtt-broker Running
```

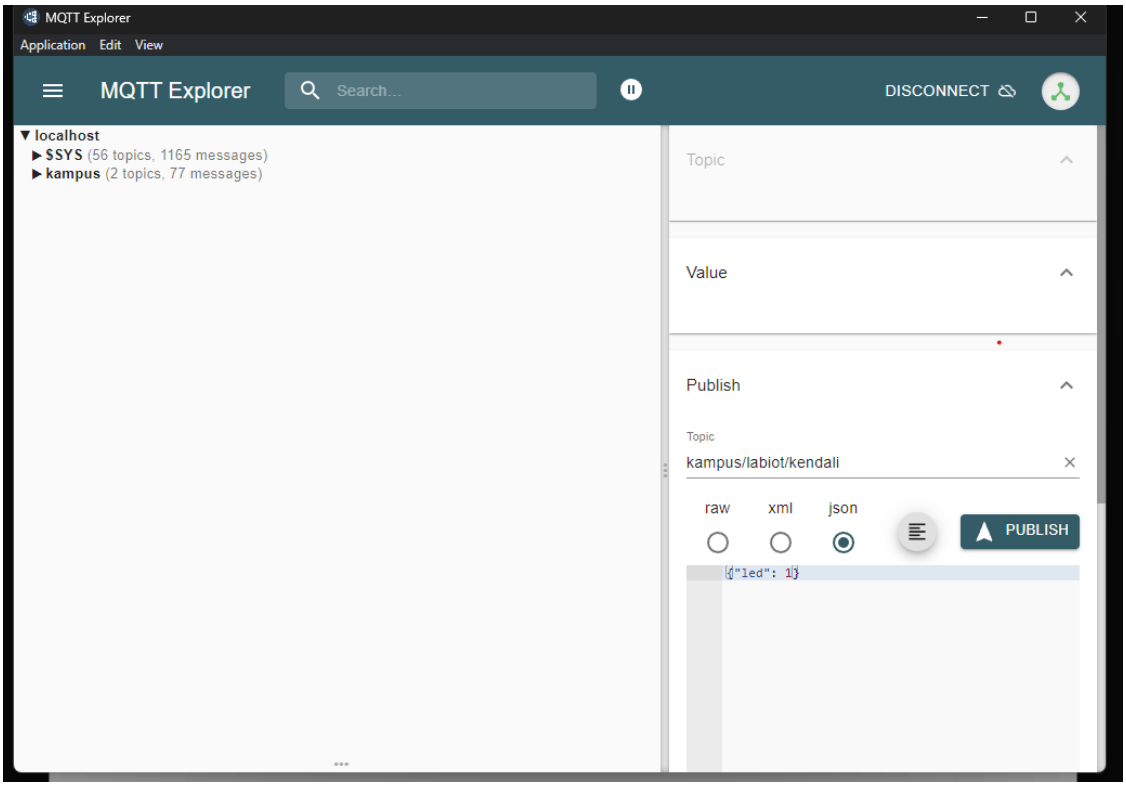




Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal

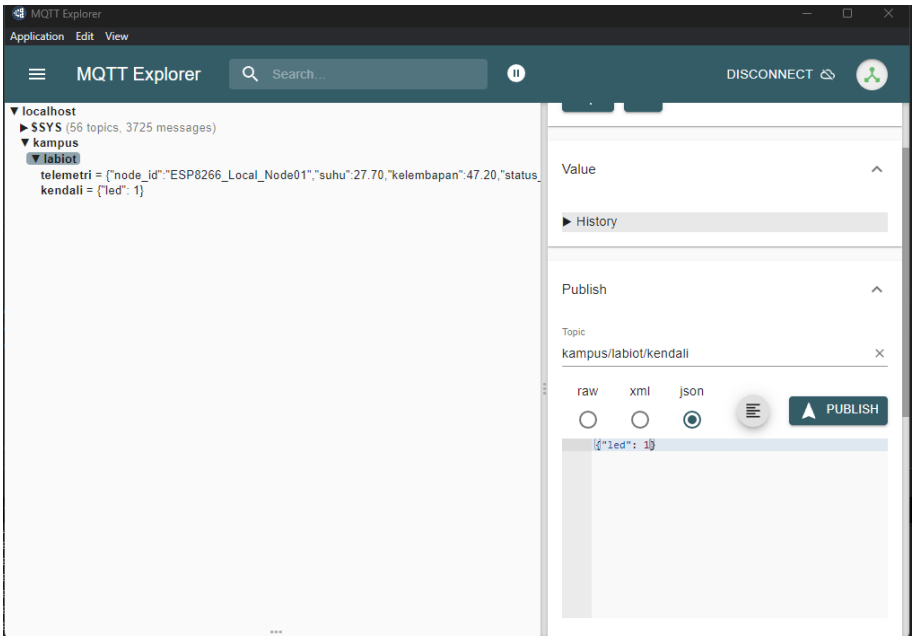


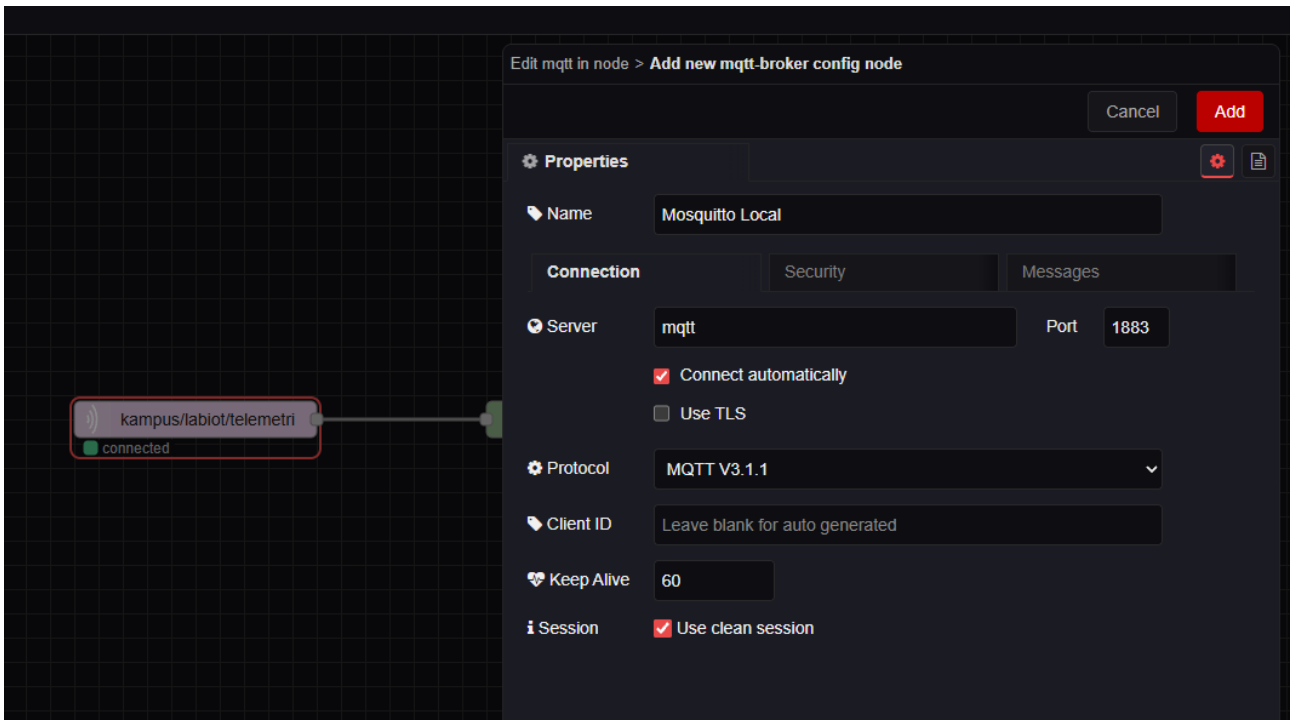


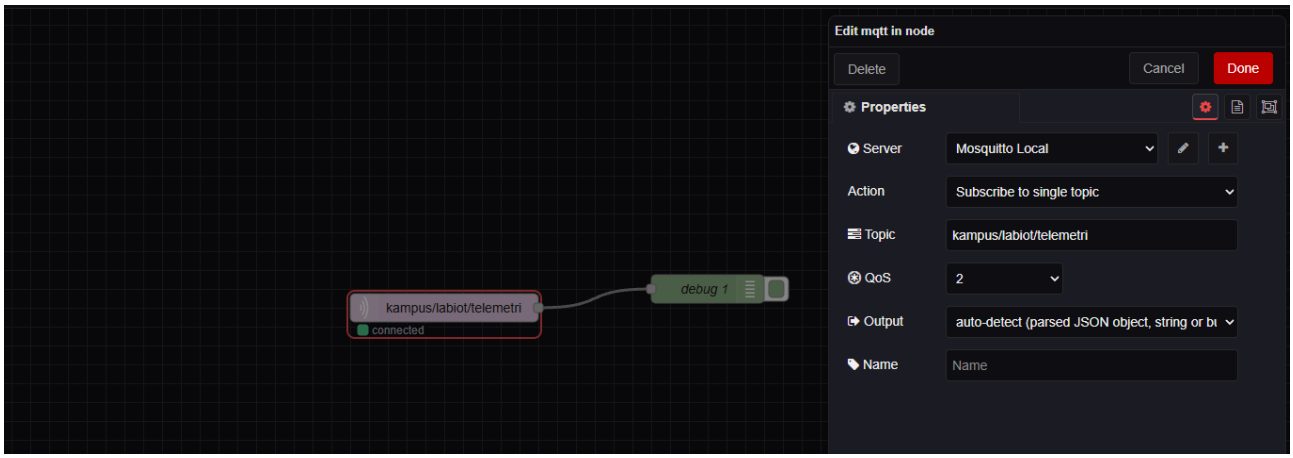
```
16:04:59.678 -> [MQTT] Menghubungkan ke Broker Lokal (10.101.94.100)... Gagal (rc=-2). Coba lagi dalam 4 detik...
16:05:11.092 -> [MQTT] Menghubungkan ke Broker Lokal (10.101.94.100)... Terhubung!
16:05:14.221 -> [MQTT] Berlangganan ke topik: kampus/labiot/kendali
16:05:14.221 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.10,"kelembapan":48.00,"status_led":1}
16:05:18.457 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.10,"kelembapan":47.90,"status_led":1}
16:05:20.912 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.10,"kelembapan":48.20,"status_led":1}
16:05:24.366 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.00,"kelembapan":48.50,"status_led":1}
16:05:28.337 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.00,"kelembapan":48.80,"status_led":1}
16:05:32.340 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.00,"kelembapan":49.00,"status_led":1}
16:05:36.339 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.00,"kelembapan":49.40,"status_led":1}
16:05:40.360 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.00,"kelembapan":49.50,"status_led":1}
```

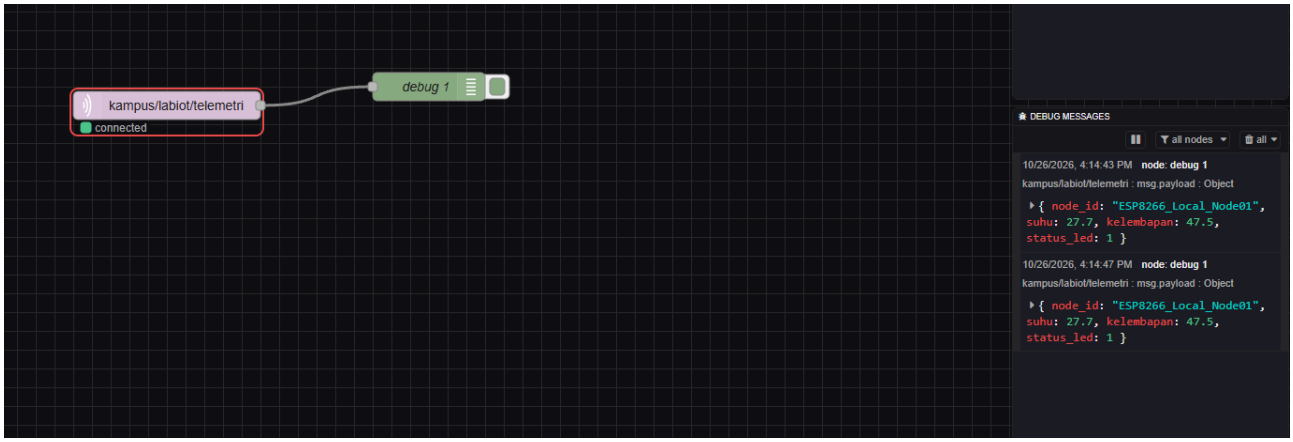
Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.




Edit mqtt in node > Add new mqtt-broker config node
Cancel
Add
Properties
Name
Mosquitto Local
Connection
Security
Messages
Server
mqtt
Port
1883
Connect automatically
Use TLS
Protocol
MQTT V3.1.1
Client ID
Leave blank for auto generated
Keep Alive
60
Session
Use clean session
kampus/labiot/telemetri
connected

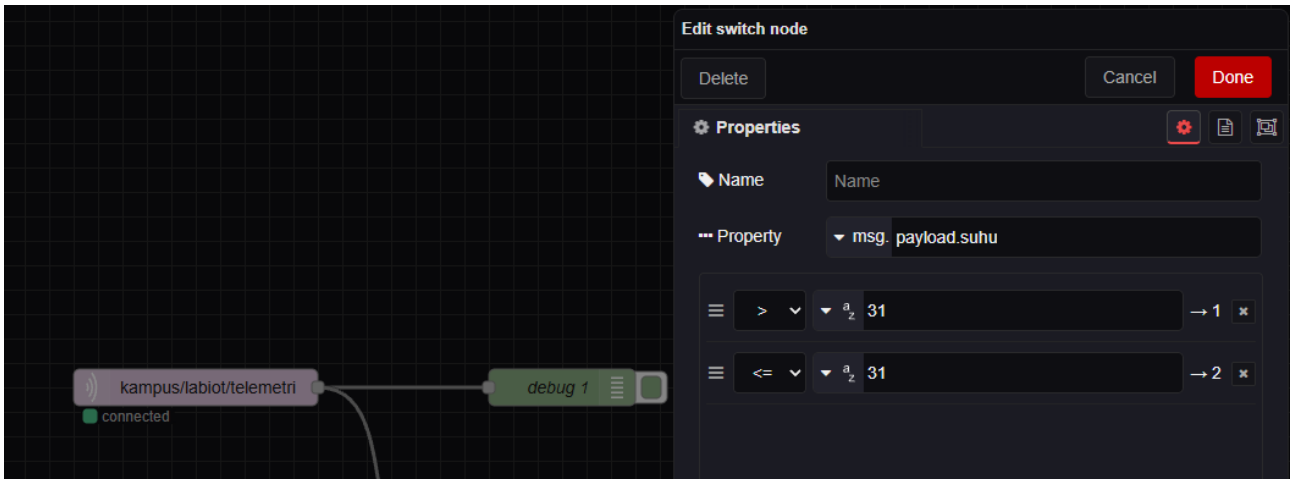

Edit mqtt in node
Delete
Cancel
Done
Properties
Server
Mosquitto Local
Action
Subscribe to single topic
Topic
kampus/labiot/telemetri
QoS
2
Output
auto-detect (parsed JSON object, string or bu
Name
Name
kampus/labiot/telemetri
connected
debug 1

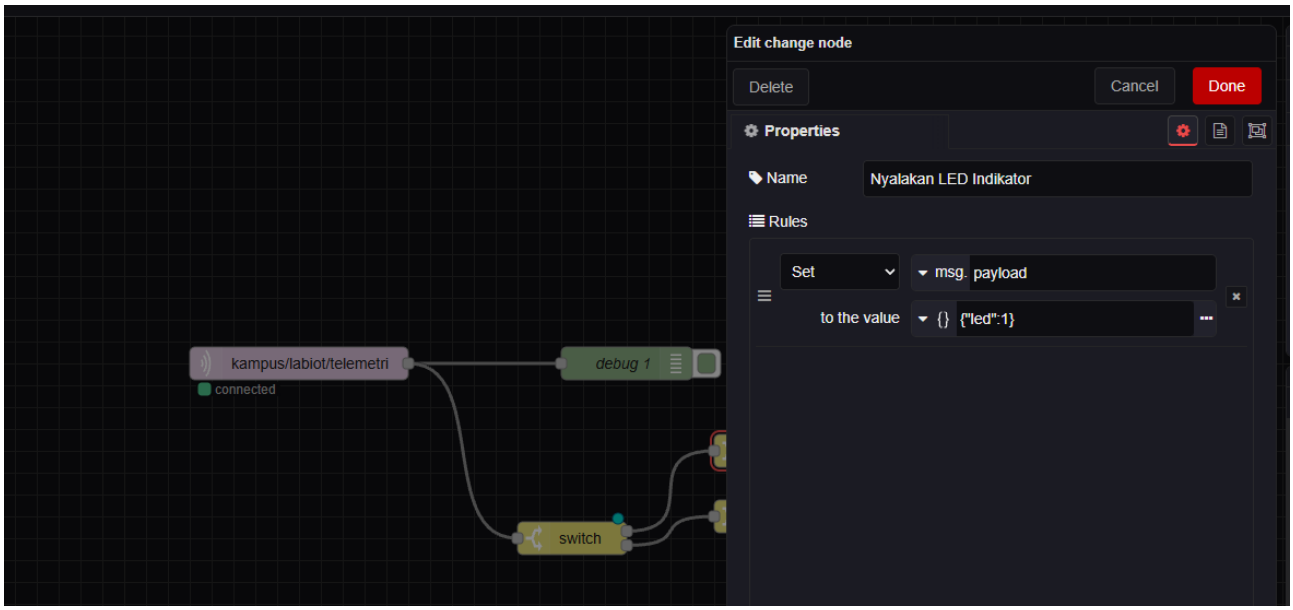

kampus/labiot/telemetri
connected
debug 1
DEBUG MESSAGES
all nodes
all
10/26/2026, 4:14:43 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.7, kelembapan: 47.5, status_led: 1 }
10/26/2026, 4:14:47 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.7, kelembapan: 47.5, status_led: 1 }

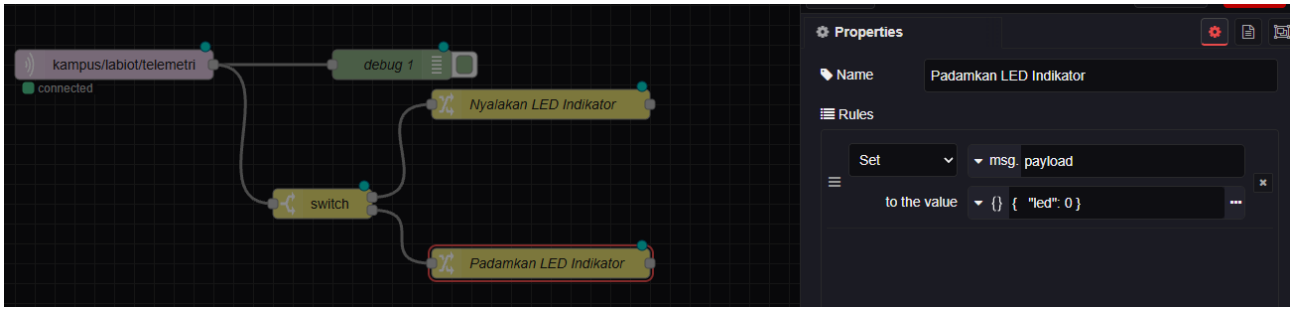
Praktikum 4 : Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

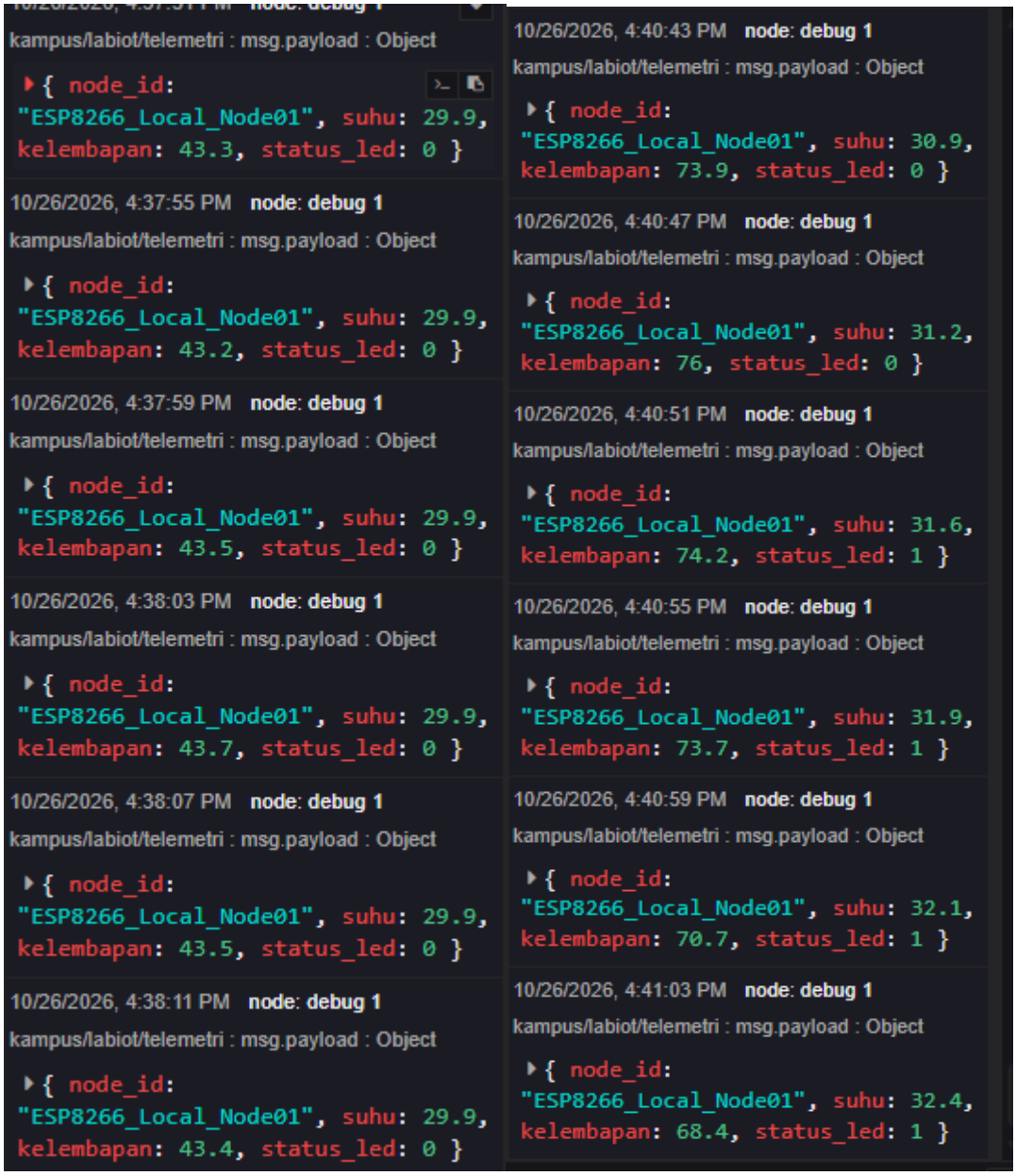
Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

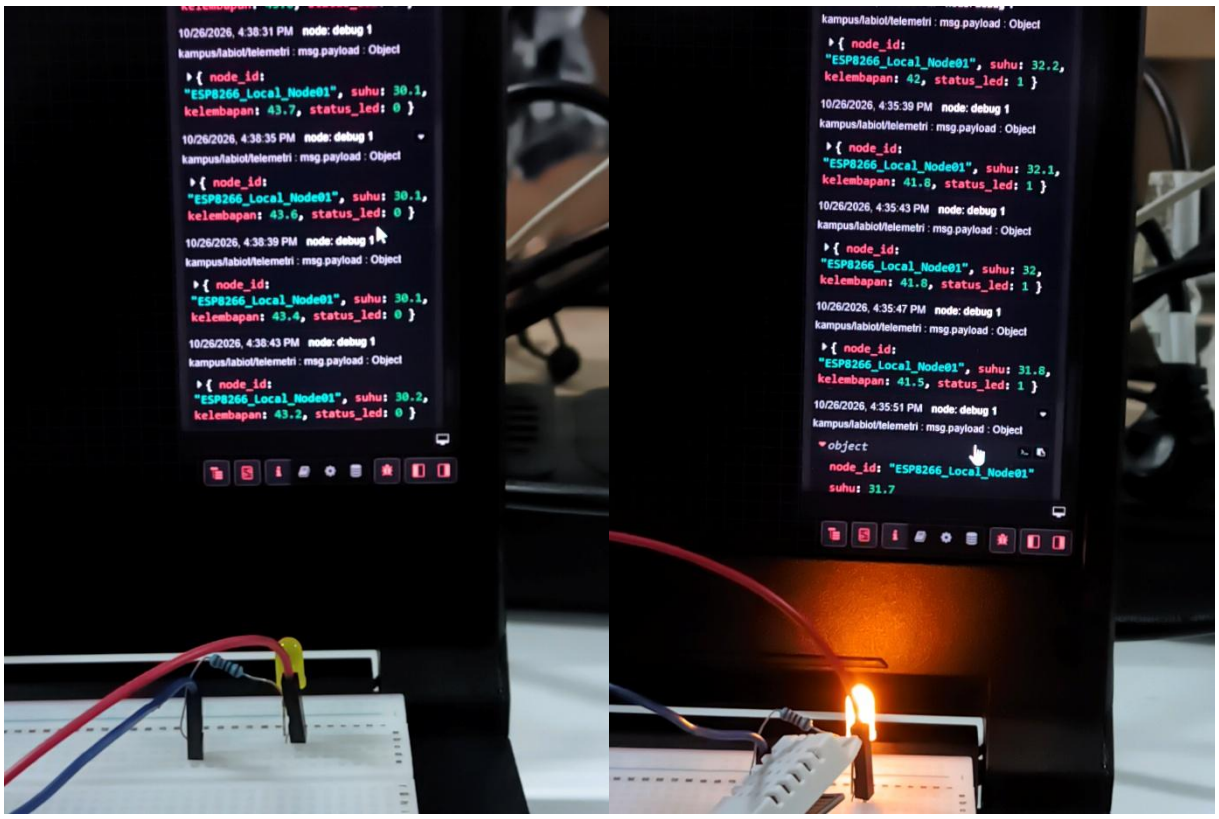
- JIKA nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1}untuk menyalakan indikator LED peringatan bahaya di pin D1.
- JIKA suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0}untuk memenuhi indikator LED.








kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 29.9, kelembapan: 43.3, status_led: 0 }
10/26/2026, 4:37:55 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 29.9, kelembapan: 43.2, status_led: 0 }
10/26/2026, 4:37:59 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 29.9, kelembapan: 43.5, status_led: 0 }
10/26/2026, 4:38:03 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 29.9, kelembapan: 43.7, status_led: 0 }
10/26/2026, 4:38:07 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 29.9, kelembapan: 43.5, status_led: 0 }
10/26/2026, 4:38:11 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 29.9, kelembapan: 43.4, status_led: 0 }
10/26/2026, 4:40:43 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 30.9, kelembapan: 73.9, status_led: 0 }
10/26/2026, 4:40:47 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.2, kelembapan: 76, status_led: 0 }
10/26/2026, 4:40:51 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.6, kelembapan: 74.2, status_led: 1 }
10/26/2026, 4:40:55 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 31.9, kelembapan: 73.7, status_led: 1 }
10/26/2026, 4:40:59 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 32.1, kelembapan: 70.7, status_led: 1 }
10/26/2026, 4:41:03 PM node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 32.4, kelembapan: 68.4, status_led: 1 }



Latihan 1: Jaringan internal Docker dan service discovery

1. Mengapa localhost menyebabkan Connection Refused

localhost (127.0.0.1) selalu menunjuk ke perangkat tempat program itu berjalan. Setiap kontainer Docker punya network namespace sendiri, lengkap dengan loopback-nya, yang terisolasi dari host dan kontainer lain. Saat Node-RED menyambung ke localhost:1883, ia mencari broker di dalam kontainernya sendiri. Di sana hanya ada proses Node-RED (port 1880), dan tidak ada proses yang mendengarkan port 1883, jadi koneksi langsung ditolak (ECONNREFUSED). Broker Mosquitto berjalan di kontainer lain dengan loopback yang berbeda. Memakai IP laptop (192.168.1.15) secara teknis bisa bekerja lewat port forwarding, tetapi rapuh karena IP itu berubah setiap kali pindah jaringan Wi-Fi.

2. Mekanisme Embedded DNS dan Bridge Network

Docker Compose otomatis membuat jaringan bridge khusus proyek, dan semua layanan di berkas Compose bergabung ke dalamnya. Tiap kontainer mendapat antarmuka virtual dengan IP privat (misalnya 172.18.0.2 dan 172.18.0.3) yang terhubung lewat switch virtual Di jaringan bridge buatan pengguna, Docker menjalankan DNS server internal di alamat 127.0.0.11, dan setiap kontainer otomatis memakainya sebagai resolver. Saat kontainer dibuat, Docker mendaftarkan nama layanan (mqtt) dan nama kontainer (mqtt-broker) beserta IP-nya ke DNS ini. Ketika Node-RED

menyambung ke mqtt:1883, resolver bertanya ke 127.0.0.11, mendapat IP kontainer broker, lalu terhubung langsung lewat bridge. Keuntungannya, jika kontainer dibuat ulang dan IP-nya berubah, nama mqtt tetap valid. Koneksi antarkontainer memakai port internal (1883), sehingga tidak bergantung pada pemetaan ports: ke host. Pemetaan itu hanya dipakai klien di luar Docker, seperti NodeMCU.

Latihan 2: Subnet dan firewall

1. Mengapa NodeMCU tidak akan berhasil memublikasikan data

NodeMCU (192.168.43.102) berada di jaringan 192.168.43.0/24, sedangkan laptop (192.168.1.15) berada di 192.168.1.0/24. Dengan subnet mask 255.255.255.0, tiga oktet pertama menentukan jaringan, dan nilainya berbeda (43 dan 1). NodeMCU menganggap laptop berada di luar jaringan lokalnya, sehingga paket harus diteruskan lewat gateway. Router rumah atau hotspot biasa tidak merutekan antar-subnet pribadi seperti ini, dan NAT membuat laptop tidak terjangkau. Akibatnya client.connect() gagal (rc=-2 atau timeout -4), tidak ada sesi MQTT yang terbentuk, dan publish() tidak pernah terjadi. Firewall laptop bahkan tidak menjadi persoalan, karena paketnya tidak pernah sampai.

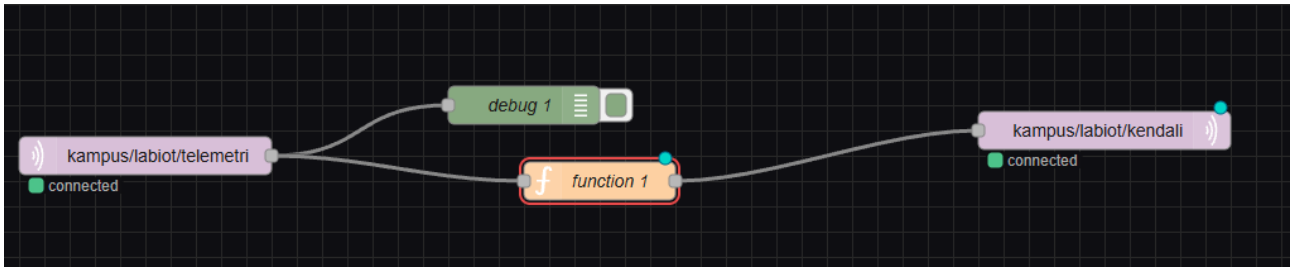
2. Langkah agar kedua perangkat berada di subnet yang sama

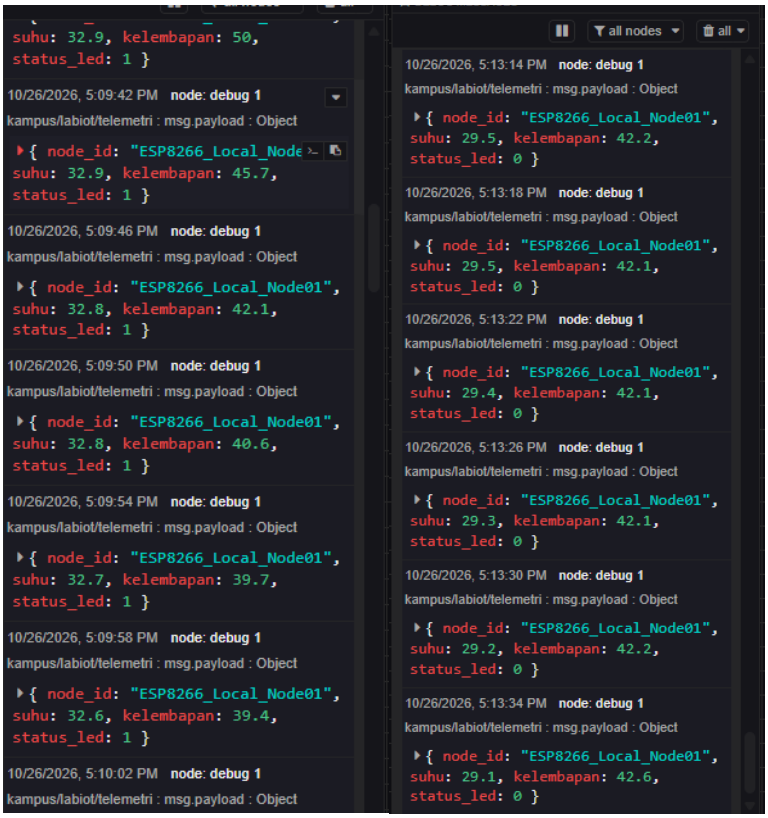
Hubungkan laptop dan NodeMCU ke jaringan Wi-Fi yang sama, misalnya laptop dipindah ke hotspot HP (192.168.43.x), atau NodeMCU diarahkan ke Wi-Fi rumah (192.168.1.x). Setelah itu, ambil IPv4 laptop yang baru lewat ipconfig dan isikan ke mqttServer di sketsa, sehingga tiga oktet pertamanya sama dengan IP NodeMCU. Terakhir, izinkan inbound TCP port 1883 di firewall laptop.

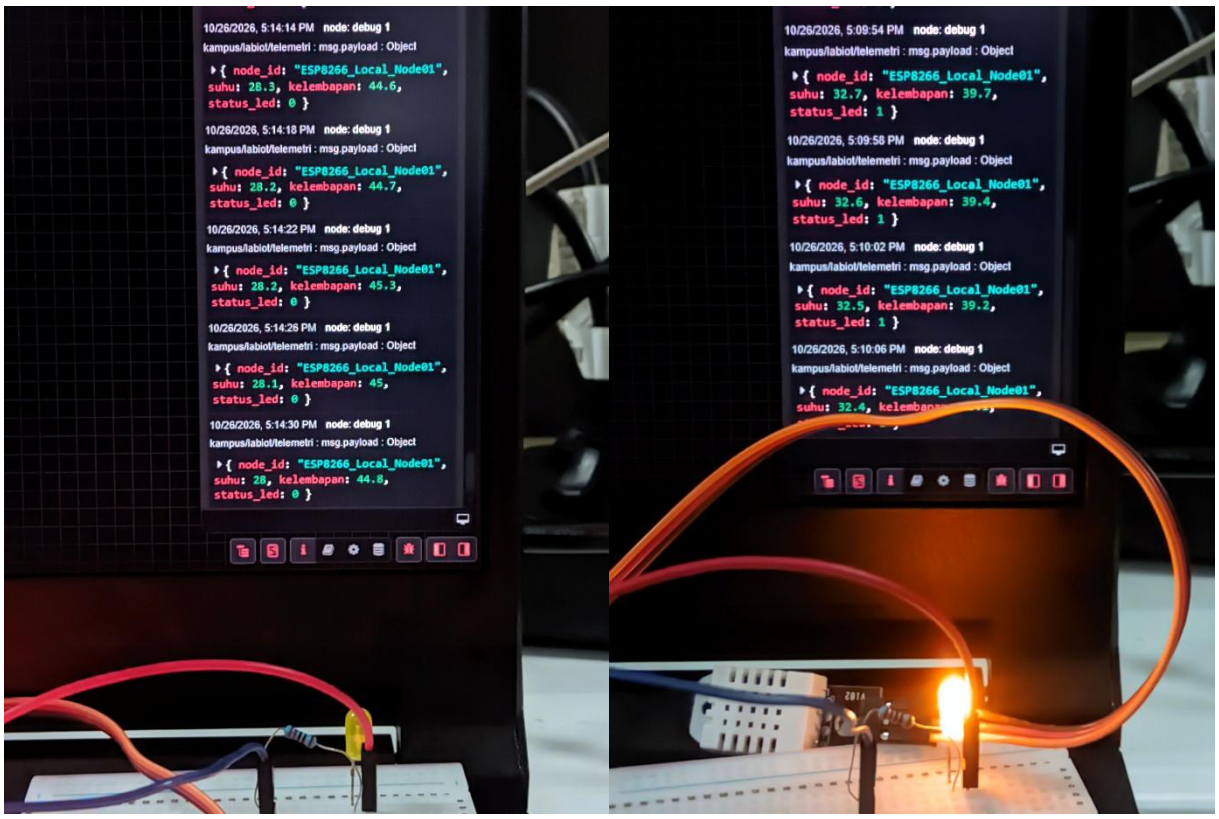
Alternatifnya, nyalakan Mobile Hotspot Windows di laptop dan sambungkan NodeMCU ke sana. NodeMCU otomatis masuk ke subnet laptop (192.168.137.x) dan alamat broker menjadi 192.168.137.1. Untuk hasil yang stabil, IP laptop bisa dibuat tetap lewat IP statis atau DHCP reservation di router.

Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas31.0°C(misal:31.05°C → 30,95°C → 31.02°C), lampu indikator LED akan berkedip-kedip cepat (*berkedip / berburu*) yang mengganggu stabilitas sistem visual.
Rancang modifikasi alur Node-RED menggunakan function nodeberbahasa JavaScript yang menerapkan logika Histeresis :
LED baru menyala jika suhu> 31,5°C(msg.payload = {"led": 1}).

LED baru padam jika suhu mendingin hingga< 30.0°C(msg.payload = {"led": 0}). Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!







Latihan 4

1. Dampak menghapus baris volume lalu docker compose down dan up lagi

Tanpa volume, alur Node-RED tersimpan di /data di dalam kontainer. Perintah down menghapus kontainer beserta isinya, jadi setelah up lagi, kanvas kembali kosong dan semua alur hilang. Pada Mosquitto, berkas persistensinya (pesan retained dan sesi klien) ikut hilang. Dengan volume, data tersimpan di folder laptop sehingga tetap ada walaupun kontainer dihapus.

2. Bind Mount vs Named Volume

- Bind mount (./nodered:/data): memetakan folder tertentu di host yang kamu tentukan sendiri. Datanya mudah dibuka dan diedit langsung, tetapi bergantung pada struktur folder host.
- Named volume (nodered_data:/data): penyimpanan yang dikelola Docker. Lebih portabel dan tidak bergantung pada folder host, tetapi tidak mudah diakses langsung dari host.

Tugas: Pengamanan Server IoT Industri (Otentikasi MQTT & Kontrol Akses)

```
D:\>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/

D:\>
```

```
# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt
```

